RestoFlow POS

Restaurant Billing, KOT, Inventory and Profit Automation
Multi-tenant SaaS Platform

Document ID	SRS-RPOS-2026-V1.0
Title	Software Requirements and Build Specification
Version / Date	1.0 / October 11, 2026
Prepared for	Antigravity (AI coding agent) - full project implementation
Stack	Go (backend), React + TypeScript (frontend), PostgreSQL, Redis (optional)
Working name	RestoFlow POS (rename freely; no third-party branding must be copied)

Table of Contents

1. Introduction 4

1.1 Purpose . . . 4

1.2 Product summary . . . 4

1.3 Goals . . . 4

1.4 Instructions to the implementing agent . . . 4

1.5 Out of scope for v1 . . . 4

2. Users and Roles 4

2.1 Permission matrix . . . 5

3. Technology Stack 5

4. System Architecture 5

4.1 High-level view . . . 5

4.2 Backend layering . . . 6

4.3 Multi-tenancy . . . 6

4.4 Security requirements . . . 6

5. Functional Requirements 6

5.1 Authentication and staff (FR-AUTH) . . . 6

5.2 Restaurant, branch and settings (FR-SET) . . . 6

5.3 Menu management (FR-MENU) . . . 7

5.4 Tables and floor (FR-TBL) . . . 7

5.5 POS and orders (FR-POS) . . . 7

5.6 Kitchen order tickets and KDS (FR-KOT) . . . 7

5.7 Billing and payments (FR-BILL) . . . 8

5.8 Shifts and cash control (FR-SHIFT) . . . 8

5.9 Inventory and recipes (FR-INV) . . . 8

5.10 Purchasing, suppliers and expenses (FR-PUR) . . . 8

5.11 Reports and dashboard (FR-REP) . . . 9

5.12 Audit log (FR-AUD) . . . 9

5.13 Subscription and platform administration (FR-SUB) . . . 9

5.14 Offline, PWA and printing (FR-OFF / FR-PRT) . . . 9

5.15 Localization and UX (FR-UX) . . . 9

6. Core Workflows and Business Rules 10

6.1 Dine-in order lifecycle . . . 10

6.2 State machines . . . 10

6.3 Key business rules . . . 10

7. Realtime Events (WebSocket) 10

8. Database Design (PostgreSQL) 11

8.1 Platform and identity . . . 11

8.2 Menu and floor . . . 11
8.3 Orders, billing and shifts . . . 12
8.4 Inventory, purchasing and finance . . . 12
9. REST API Specification **13**
9.1 Auth and users . . . 13
9.2 Setup and menu . . . 13
9.3 Orders, KOT and billing . . . 14
9.4 Shifts, inventory and purchasing . . . 14
9.5 Reports, audit and platform admin . . . 15
10. Frontend Specification **15**
10.1 Applications inside one React project . . . 15
10.2 Key screens . . . 15
10.3 Frontend structure . . . 16
10.4 UX requirements . . . 16
11. Backend Project Structure **16**
12. Offline Sync Design **16**
13. Non-Functional Requirements **17**
14. Phased Implementation Plan **17**
Phase 0 - Foundation (week 1) . . . 17
Phase 1 - MVP billing (weeks 2-4) . . . 17
Phase 2 - Kitchen and operations (weeks 5-6) . . . 17
Phase 3 - Inventory and profit (weeks 7-9) . . . 18
Phase 4 - Platform, multi-branch, offline (weeks 10-13) . . . 18
Phase 5 - Hardening and launch (weeks 14-15) . . . 18
15. Testing and Quality **18**
16. Deployment and Configuration **19**
16.1 Environment variables . . . 19
16.2 Delivery steps . . . 19
17. Definition of Done and Seed Data **19**
Appendix A. Calculation Rules and Test Vectors **19**
A.1 Bill calculation (exclusive tax mode) . . . 19
A.2 Test vectors (BDT, amounts in taka for readability) . . . 20
A.3 Profit and loss . . . 20
A.4 Weighted average cost . . . 20
A.5 Receipt layout (80mm / 58mm) . . . 20

1. Introduction

1.1 Purpose

This document is the complete specification for building **RestoFlow POS**, a cloud-based Restaurant Point of Sale and management system. It is written so that an AI coding agent (Antigravity) can implement the entire project end to end: database, Go backend API, React frontend, realtime kitchen display, offline support, reports, subscription handling and deployment.

1.2 Product summary

RestoFlow POS helps restaurant owners run daily operations and, most importantly, **know their real profit**. It covers fast billing and receipt printing, kitchen order tickets (KOT) with a kitchen display screen, table management, stock and wastage control with an audit trail, and automatic profit reports. It is sold as a subscription (SaaS) with a free trial, and targets restaurants in Bangladesh first, then Saudi Arabia, UAE, Kuwait and Sri Lanka.

1.3 Goals

- Bill a customer and print a receipt in a few seconds, with minimal taps.
- Send orders to the kitchen instantly (KOT print and live Kitchen Display).
- Track stock automatically from recipes; reduce wastage and theft with approvals and audit logs.
- Show the owner daily and monthly net profit without manual calculation.
- Keep working when internet drops (offline-first POS) and sync later.
- Support Bangla and English UI, multiple currencies, multi-branch and multi-tenant isolation.

1.4 Instructions to the implementing agent

> Build the system in the phases defined in Section 14. Finish and test each phase before starting the next. Follow the data model (Section 8), API contract (Section 9) and business rules (Section 6 and Appendix A) exactly. When a detail is not specified, choose the simplest secure option, document the decision in /docs/decisions.md, and continue. Never store money as floating point. Every tenant-owned table must carry restaurant_id and every query must filter by it.

1.5 Out of scope for v1

- Online ordering website or customer-facing mobile app.
- Third-party delivery platform integrations (Foodpanda, Pathao Food, etc.).
- Automated subscription payment gateway (v1 uses manual activation by super admin; hooks are prepared).
- Accounting export to external ERPs, payroll and HR.

2. Users and Roles

Role	Scope	Typical actions
Super Admin	Platform (all tenants)	Manage restaurants, plans, trials, suspensions, support impersonation (audited)
Owner	One restaurant, all branches	Settings, menu, pricing, staff, all reports, subscription view
Manager	Assigned branch(es)	Approvals (void, discount over limit, wastage), stock, purchases, shift review, branch reports
Cashier	Assigned branch	Billing, payments, shift open/close, receipt reprint (audited)
Waiter	Assigned branch	Tables, create and edit orders, send to kitchen, request bill
Kitchen	Assigned branch/station	View KOT on KDS, update item status

2.1 Permission matrix

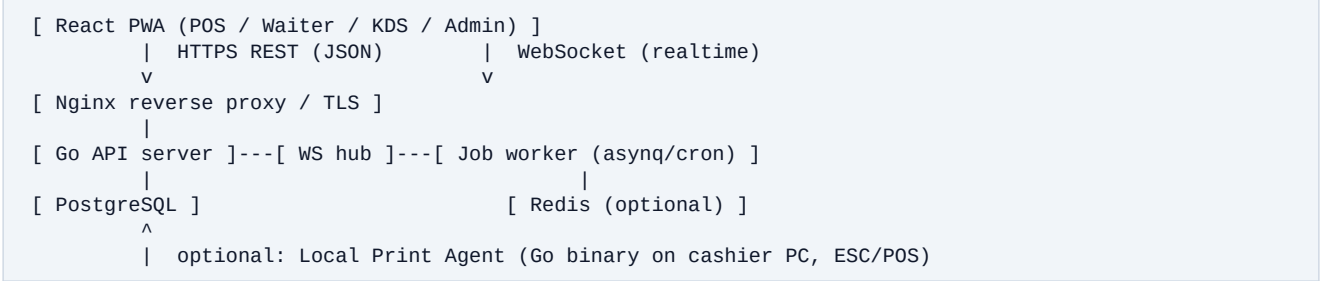
Capability	Owner	Manager	Cashier	Waiter	Kitchen
Create / edit order	Yes	Yes	Yes	Yes	No
Send KOT	Yes	Yes	Yes	Yes	No
Update KOT item status	Yes	Yes	No	No	Yes
Take payment / print bill	Yes	Yes	Yes	No	No
Apply discount up to limit	Yes	Yes	Yes (limit)	No	No
Void item after KOT / cancel order	Yes	Yes (approve)	Request	Request	No
Open / close shift	Yes	Yes	Yes	No	No
Manage menu and prices	Yes	Yes	No	No	No
Stock in / adjust / wastage	Yes	Yes	No	No	No
View profit reports	Yes	Branch only	No	No	No
Manage staff and settings	Yes	No	No	No	No

3. Technology Stack

Layer	Choice	Notes
Backend language	Go 1.22+	Modular monolith; clean layering (handler, service, repository)
HTTP router	chi (or gin)	Middleware for auth, tenant, rate limit, request ID, logging
Database	PostgreSQL 15+	Single database, shared schema, restaurant_id on every tenant table
DB access	pgx + sqlc	Migrations with golang-migrate or goose
Cache / jobs	Redis + asynq (optional)	Background jobs; fallback to in-process cron if Redis is absent
Realtime	WebSocket (gorilla or nhooyr)	Rooms per restaurant and branch; KDS and table updates
Auth	JWT access + refresh tokens	Argon2id or bcrypt password hashing; PIN login for cashier/waiter
Frontend	React 18 + TypeScript + Vite	PWA via vite-plugin-pwa; touch-first UI
UI / styling	Tailwind CSS + shadcn/ui	Large touch targets (min 44px), dark and light themes
State and data	TanStack Query + Zustand	Zustand for cart and POS session; Query for server data
Offline storage	IndexedDB via Dexie	Outbox queue and cached menu/tables
Charts	Recharts	Dashboard and reports
i18n	react-i18next	English and Bangla; locale-aware number and date formats
Printing	Browser print (58/80mm CSS); later Go print agent	ESC/POS over TCP or USB for direct thermal printing
Deployment	Docker + docker-compose, Nginx	CI with GitHub Actions

4. System Architecture

4.1 High-level view



4.2 Backend layering

- **handler**: HTTP parsing, validation, response mapping only.
- **service**: business rules, transactions, permission checks, event emission.
- **repository**: SQL (sqlc) only; no business logic.
- **realtime**: WebSocket hub that publishes domain events (order.created, kot.updated, table.updated).
- **jobs**: low-stock alerts, daily report snapshot, subscription expiry, stale-order cleanup.

4.3 Multi-tenancy

- Shared database, shared schema. Every tenant table has restaurant_id (UUID, NOT NULL, indexed).
- Tenant is derived from the JWT claim, never from client-supplied parameters.
- A repository helper must always add the restaurant_id predicate. Add automated tests that prove tenant A can never read or write tenant B data.
- Optional hardening: PostgreSQL Row Level Security with SET LOCAL app.restaurant_id per transaction.

4.4 Security requirements

- Passwords hashed with Argon2id (or bcrypt cost 12). PINs (4-6 digits) hashed and rate limited.
- Access token 15 minutes, refresh token 30 days with rotation and revocation list.
- Role-based middleware on every route; deny by default.
- Rate limiting on login and public endpoints; account lockout after repeated failures.
- All sensitive actions (void, discount, price change, stock adjust, refund, reprint) written to audit_logs.
- Input validation on all endpoints; parameterized SQL only; strict CORS; security headers.
- Secrets from environment variables only; never commit .env files.

5. Functional Requirements

Requirement IDs (FR-xxx) are referenced by the phase plan and acceptance tests. Priority: M = must, S = should, C = could.

5.1 Authentication and staff (FR-AUTH)

ID	Requirement	Pri
FR-AUTH-1	Owner registers a restaurant (name, phone, email, password) and starts a trial automatically.	M
FR-AUTH-2	Login by email and password; refresh token flow; logout revokes refresh token.	M
FR-AUTH-3	Quick PIN login on POS terminals for cashier and waiter, scoped to a device and branch.	S
FR-AUTH-4	Owner/Manager can create, edit, deactivate users and assign role and branches.	M
FR-AUTH-5	Password reset by email/SMS OTP (provider abstracted behind an interface).	S
FR-AUTH-6	Per-role discount limit (percent) configurable per user.	S

5.2 Restaurant, branch and settings (FR-SET)

ID	Requirement	Pri
FR-SET-1	Restaurant profile: name, logo, address, phone, tax number, currency, timezone, language.	M
FR-SET-2	Multiple branches; each has its own tables, stock, shifts, printers and staff assignment.	M
FR-SET-3	Tax settings: VAT %, service charge %, inclusive/exclusive mode, rounding rule.	M
FR-SET-4	Receipt template: header, footer text, logo, paper width (58mm or 80mm), show/hide VAT number.	M
FR-SET-5	Payment methods enabled per restaurant: cash, card, bKash, Nagad, Rocket, bank, custom.	M
FR-SET-6	Kitchen stations (e.g., Grill, Fry, Beverage) and mapping of menu categories to stations.	S

5.3 Menu management (FR-MENU)

ID	Requirement	Pri
FR-MENU-1	Categories (ordering, color, icon) and items (name EN/BN, price, image, SKU, tax class, active flag).	M
FR-MENU-2	Variants (size: small, medium, large) with own price and recipe.	M
FR-MENU-3	Add-on groups (extra cheese, sauces) with min/max selection and price.	S
FR-MENU-4	Item availability toggle (86 an item) that takes effect instantly on all POS terminals.	M
FR-MENU-5	Branch-level price override and availability.	C
FR-MENU-6	Bulk import/export of menu via CSV.	S
FR-MENU-7	Combo/set meals composed of items.	C

5.4 Tables and floor (FR-TBL)

ID	Requirement	Pri
FR-TBL-1	Define sections (Ground, Rooftop, VIP) and tables with seat count and layout position.	M
FR-TBL-2	Table statuses: free, occupied, bill_requested, reserved, cleaning, with live color coding.	M
FR-TBL-3	Transfer an order to another table; merge two tables; split a bill by item or by equal share.	S
FR-TBL-4	Basic reservation (name, phone, time, party size) that marks a table reserved.	C

5.5 POS and orders (FR-POS)

ID	Requirement	Pri
FR-POS-1	Order types: dine_in, takeaway, delivery. Dine-in requires a table; takeaway/delivery get a daily token number.	M
FR-POS-2	Fast item selection from category tabs and search; quantity, variant, add-ons, per-item notes.	M
FR-POS-3	Cart supports hold (park) and resume of multiple orders per terminal.	S
FR-POS-4	Orders get a client-generated UUID so they can be created offline; server order_no assigned on sync.	M
FR-POS-5	Add items to an existing open order; each addition becomes a new KOT batch.	M
FR-POS-6	Remove/void item: free before KOT; after KOT requires reason and manager approval; logged.	M
FR-POS-7	Customer record (name, phone, address) attachable to an order; delivery fee and rider assignment.	S
FR-POS-8	Order-level and item-level discount (percent or fixed) with reason; limit by role.	M
FR-POS-9	Keyboard and barcode scanner support for fast entry on desktop.	C

5.6 Kitchen order tickets and KDS (FR-KOT)

ID	Requirement	Pri
FR-KOT-1	Send to Kitchen creates a KOT with only the new items, table, waiter, time and notes.	M
FR-KOT-2	KOT print (browser/print agent) per station and/or live Kitchen Display Screen via WebSocket.	M
FR-KOT-3	Item status flow: pending, preparing, ready, served (cancelled allowed with reason).	M
FR-KOT-4	KDS shows elapsed time, highlights late tickets (configurable threshold) and plays a sound on new ticket.	S
FR-KOT-5	Waiter receives a notification when items are ready.	S
FR-KOT-6	Reprint of KOT is allowed and audited.	M

5.7 Billing and payments (FR-BILL)

ID	Requirement	Pri
FR-BILL-1	Generate bill from order using the calculation rules in Appendix A; show full breakdown.	M
FR-BILL-2	Multiple payments per bill (split tender): cash, card, mobile banking, with reference number.	M
FR-BILL-3	Cash received and change due calculation; due/credit sale for registered customers (S).	M
FR-BILL-4	Print receipt (58/80mm), reprint (audited), and share as PDF or WhatsApp link (S).	M
FR-BILL-5	On full payment: order closed, table freed, inventory deducted, sales recorded in one transaction.	M
FR-BILL-6	Refund or cancel a paid bill only by Manager/Owner with reason; stock restore option.	S
FR-BILL-7	Sequential, gap-free invoice number per branch per day or fiscal year (configurable prefix).	M

5.8 Shifts and cash control (FR-SHIFT)

ID	Requirement	Pri
FR-SHIFT-1	Cashier opens a shift with opening cash; only one open shift per cashier per terminal.	M
FR-SHIFT-2	Cash in/out entries (petty cash, bank deposit) with reason.	S
FR-SHIFT-3	Close shift: counted cash vs expected cash, show variance, produce a shift report.	M
FR-SHIFT-4	Cannot close a shift while it has unpaid open orders unless transferred or acknowledged.	S

5.9 Inventory and recipes (FR-INV)

ID	Requirement	Pri
FR-INV-1	Inventory items (ingredients and packaged goods) with unit, unit conversions, min level, category.	M
FR-INV-2	Recipes: menu item/variant maps to inventory items with quantity per portion.	M
FR-INV-3	Automatic stock deduction on paid order (configurable: on KOT send or on payment) via stock_movements.	M
FR-INV-4	Stock-in via purchase receiving; weighted-average cost recalculated on each receipt.	M
FR-INV-5	Wastage and manual adjustment entries require a reason; approval above a configurable value.	M
FR-INV-6	Low-stock alerts (dashboard, push, optional SMS/email) via background job.	S
FR-INV-7	Stock count (physical audit) session that generates variance adjustments.	S
FR-INV-8	Inter-branch stock transfer.	C

5.10 Purchasing, suppliers and expenses (FR-PUR)

ID	Requirement	Pri
FR-PUR-1	Supplier directory with contact and outstanding balance.	S
FR-PUR-2	Purchase order lifecycle: draft, ordered, received (partial allowed), cancelled.	S
FR-PUR-3	Receiving a PO creates stock-in movements and supplier payable.	S
FR-PUR-4	Operating expenses (rent, salary, utilities, other) by category, date and branch.	M
FR-PUR-5	Supplier payments recorded against payables.	C

5.11 Reports and dashboard (FR-REP)

ID	Report	Pri
FR-REP-1	Owner dashboard: today and month sales, orders, average bill, net profit, top items, low stock, open shifts.	M
FR-REP-2	Sales report by day, week, month; by branch, order type, payment method, category, item, staff.	M
FR-REP-3	Profit and loss: Revenue - COGS - Expenses = Net profit (formula in Appendix A).	M
FR-REP-4	Item performance: quantity, revenue, cost, margin; best and worst sellers.	M
FR-REP-5	Discount, void, refund and reprint report (theft control) with staff and reason.	M
FR-REP-6	Stock report: current stock, valuation, movements, wastage, variance.	M
FR-REP-7	Shift and cash report; VAT/tax report for filing.	M
FR-REP-8	Export any report to CSV and PDF; date-range filters with presets.	S
FR-REP-9	Daily summary snapshot job and optional daily email/WhatsApp message to the owner.	C

5.12 Audit log (FR-AUD)

- Immutable audit_logs record: who, when, action, entity, before/after JSON, IP, device.
- Searchable by user, action, entity and date; visible to Owner and Manager.
- Logged actions include login failures, price changes, discounts, voids, cancellations, refunds, reprints, stock adjustments, wastage, user and role changes, settings changes, super admin impersonation.

5.13 Subscription and platform administration (FR-SUB)

ID	Requirement	Pri
FR-SUB-1	New restaurant gets a trial (default 14 days, configurable) with full features.	M
FR-SUB-2	Plans (Starter, Pro, Enterprise) define limits: branches, users, features (inventory, multi-branch, API).	M
FR-SUB-3	Expired trial/subscription puts the restaurant in read-only grace period, then suspended.	M
FR-SUB-4	Super admin console: list tenants, activate/extend/suspend, change plan, view usage, impersonate (audited).	M
FR-SUB-5	Subscription invoices (manual payment recording in v1; payment gateway hook interface prepared).	S
FR-SUB-6	In-app banners for trial days left and renewal reminders.	S

5.14 Offline, PWA and printing (FR-OFF / FR-PRT)

ID	Requirement	Pri
FR-OFF-1	Installable PWA; menu, tables and settings cached in IndexedDB; app shell cached by service worker.	M
FR-OFF-2	When offline, POS can create orders, add items, take payments and print receipts; writes go to an outbox.	S
FR-OFF-3	Outbox syncs in order when online using idempotency keys; conflicts reported in a sync-issues screen.	S
FR-OFF-4	Visible online/offline/syncing indicator on every POS screen.	M
FR-PRT-1	Receipt and KOT print via browser print with 58mm and 80mm CSS templates.	M
FR-PRT-2	Local print agent (Go binary) exposing localhost API to send ESC/POS to network or USB printers; supports cash drawer kick and multiple printers (bill printer, kitchen printer per station).	S

5.15 Localization and UX (FR-UX)

- English and Bangla UI with runtime language switch; Bangla digits option; currency symbol and formatting per restaurant.
- Touch-first POS layout, large buttons, no hover-only interactions; works on 10-inch tablets and desktop.
- Light and dark themes; high contrast for kitchen screens.
- Fast: POS screen interactive in under 2 seconds on mid-range tablet; item add under 100 ms perceived.

6. Core Workflows and Business Rules

6.1 Dine-in order lifecycle

```
Table selected -> Order(draft) -> add items -> [Send to Kitchen] -> KOT created
   -> kitchen: pending -> preparing -> ready -> waiter serves (served)
   -> more items? -> new KOT batch (loop)
   -> [Request Bill] (table: bill_requested) -> discounts / split
   -> payment(s) -> Order(paid) -> table(free/cleaning) -> inventory deducted
   -> receipt printed -> shift totals updated
```

6.2 State machines

Entity	States and allowed transitions
Order	draft -> placed -> in_progress -> served -> billed -> paid. cancelled possible from draft/placed/in_progress (after KOT needs manager approval). refunded from paid (manager).
KOT item	pending -> preparing -> ready -> served. cancelled from pending/preparing with reason.
Table	free -> occupied (order created) -> bill_requested -> cleaning -> free. reserved <-> free.
Shift	open -> closed (immutable after close except manager correction note).
Purchase order	draft -> ordered -> partially_received -> received. cancelled from draft/ordered.
Subscription	trial -> active -> past_due (grace read-only) -> suspended. active -> cancelled.

6.3 Key business rules

- Paid orders are immutable. Corrections happen through refund or adjustment records, never edits.
- Every payment write, stock movement and order status change happens inside a database transaction.
- Invoice numbers are gap-free per branch and generated inside the payment transaction using a counter row with SELECT ... FOR UPDATE.
- Voiding an item after KOT sends a cancellation KOT to the kitchen and requires a reason and approval.
- Discounts above the user limit require manager approval via PIN or approval request.
- Closing a shift is blocked if the cash variance reason is missing when variance is not zero.
- Inventory never goes negative silently: allow negative stock only if the setting is enabled, and flag it in reports.
- Menu price changes do not alter historical orders: order lines snapshot name, price, tax and cost at sale time.

7. Realtime Events (WebSocket)

Endpoint: **GET /ws?token=JWT&branch_id=UUID**. The server joins the client to rooms restaurant:{id} and branch:{id}. Messages are JSON: {event, data, ts}.

Event	Published when	Consumers
order.created / order.updated	Order created or items changed	POS, waiter, manager
kot.created	Send to Kitchen	KDS, kitchen printer service
kot.item_status	Kitchen changes an item status	Waiter app, POS, KDS
table.updated	Table status changes	All POS and waiter devices
menu.availability	Item toggled unavailable	All POS devices
stock.low	Item falls below minimum	Manager and owner dashboard
shift.updated	Shift opened or closed	Manager dashboard
subscription.warning	Trial/plan nearing expiry or suspended	Owner UI

8. Database Design (PostgreSQL)

Conventions: UUID primary keys (uuid v7 preferred); created_at, updated_at TIMESTAMPTZ on all tables; soft delete via deleted_at where history matters; money as **BIGINT in minor units** (poisha/fils); quantities as NUMERIC(14,3); every tenant table has restaurant_id and an index starting with restaurant_id.

8.1 Platform and identity

Table	Key columns	Notes
restaurants	id, name, slug, phone, email, address, logo_url, currency, timezone, language, tax_no, status	Tenant root
branches	id, restaurant_id, name, address, phone, is_active, invoice_prefix	
users	id, restaurant_id (null for super admin), name, email, phone, password_hash, pin_hash, role, discount_limit_pct, is_active, last_login_at	Unique email per platform
user_branches	user_id, branch_id	Staff-to-branch assignment
refresh_tokens	id, user_id, token_hash, expires_at, revoked_at, device	Rotation
plans	id, code, name, price_minor, interval, max_branches, max_users, features JSONB	
subscriptions	id, restaurant_id, plan_id, status, trial_ends_at, current_period_end, grace_until	
subscription_invoices	id, restaurant_id, amount_minor, status, paid_at, method, reference	Manual in v1
settings	restaurant_id, key, value JSONB	Tax, receipt, stock rules, thresholds
payment_methods	id, restaurant_id, code, label, is_active, requires_reference	

8.2 Menu and floor

Table	Key columns	Notes
categories	id, restaurant_id, name_en, name_bn, sort, color, station_id, is_active	
menu_items	id, restaurant_id, category_id, name_en, name_bn, sku, base_price_minor, tax_class, image_url, is_available, is_active	
item_variants	id, item_id, name, price_minor, sku	Size etc.
addon_groups / addons	group: id, restaurant_id, name, min, max. addon: id, group_id, name, price_minor, inventory link	Linked via item_addon_groups
branch_item_overrides	branch_id, item_id, price_minor, is_available	Optional
stations	id, branch_id, name, printer_id	Kitchen stations
table_sections	id, branch_id, name, sort	
tables	id, branch_id, section_id, name, seats, pos_x, pos_y, status, current_order_id	
customers	id, restaurant_id, name, phone, address, notes, credit_limit_minor, balance_minor	

8.3 Orders, billing and shifts

Table	Key columns	Notes
orders	id (client UUID), restaurant_id, branch_id, order_no, token_no, type, table_id, customer_id, waiter_id, status, subtotal_minor, discount_minor, service_charge_minor, tax_minor, delivery_fee_minor, round_off_minor, total_minor, paid_minor, notes, opened_at, closed_at, shift_id	Unique (branch_id, order_no)
order_items	id, order_id, item_id, variant_id, name_snapshot, qty, unit_price_minor, discount_minor, tax_minor, cost_minor_snapshot, note, status, kot_id, voided_by, void_reason	Snapshot fields
order_item_addons	id, order_item_id, addon_id, name_snapshot, price_minor, qty	
kots	id, order_id, branch_id, station_id, batch_no, printed_at, created_by	
payments	id, order_id, method_code, amount_minor, reference, received_by, shift_id, paid_at	Multiple per order
invoices	id, branch_id, order_id, invoice_no, issued_at, snapshot JSONB	Gap-free via counters
refunds	id, order_id, amount_minor, reason, approved_by, restock BOOL	
shifts	id, branch_id, cashier_id, terminal_id, opened_at, closed_at, opening_cash_minor, expected_cash_minor, counted_cash_minor, variance_minor, note	
cash_movements	id, shift_id, type (in/out), amount_minor, reason	
counters	branch_id, name, date, value	Order no, token no, invoice no

8.4 Inventory, purchasing and finance

Table	Key columns	Notes
inventory_items	id, restaurant_id, name, unit, min_level, category, avg_cost_minor	
branch_stock	branch_id, inventory_item_id, quantity	Current on-hand per branch
recipes	id, menu_item_id, variant_id, inventory_item_id, qty_per_portion, waste_pct	
stock_movements	id, branch_id, inventory_item_id, type (opening, purchase, sale, wastage, adjustment, transfer_in, transfer_out, refund_restock), qty, unit_cost_minor, ref_type, ref_id, reason, created_by	Append-only ledger
stock_counts / stock_count_lines	session, expected, counted, variance	
suppliers	id, restaurant_id, name, phone, balance_minor	
purchase_orders / purchase_order_lines	supplier_id, branch_id, status, lines(item, qty, unit_cost, received_qty)	
expenses	id, branch_id, category, amount_minor, date, note, created_by	
audit_logs	id, restaurant_id, user_id, action, entity, entity_id, before JSONB, after JSONB, ip, device, created_at	Append-only
daily_snapshots	branch_id, date, sales_minor, cogs_minor, expenses_minor, orders, data JSONB	Report speed-up
outbox_sync_log	id, restaurant_id, idempotency_key, status, response JSONB	Idempotent offline sync

Required indexes

- orders (restaurant_id, branch_id, opened_at DESC), (restaurant_id, status), (table_id) where status open.
- order_items (order_id); payments (order_id), (shift_id); stock_movements (branch_id, inventory_item_id, created_at).
- audit_logs (restaurant_id, created_at DESC), (restaurant_id, entity, entity_id).
- Unique: (branch_id, invoice_no), (branch_id, order_no), (restaurant_id, sku) where not null.

9. REST API Specification

Base path **/api/v1**. JSON only. Auth via **Authorization: Bearer JWT**. Standard error body: {error: {code, message, details}}. Pagination: ?page=&page_size= returning {data, meta}. Mutating sync endpoints accept **Idempotency-Key**. Timestamps are ISO-8601 UTC.

9.1 Auth and users

Method	Path	Purpose	Roles
POST	/auth/register	Create restaurant, owner and trial	Public
POST	/auth/login	Email/password login	Public
POST	/auth/pin-login	PIN login on device	Public (device-bound)
POST	/auth/refresh, /auth/logout	Rotate or revoke tokens	Any
GET/POST/PATCH	/users, /users/{id}	Staff CRUD, activate/deactivate	Owner, Manager
GET/PATCH	/me	Current user profile	Any

9.2 Setup and menu

Method	Path	Purpose	Roles
GET/PATCH	/restaurant, /settings	Profile and settings	Owner
CRUD	/branches	Branch management	Owner
CRUD	/categories, /items, /items/{id}/variants	Menu management	Owner, Manager
PATCH	/items/{id}/availability	Toggle availability (realtime)	Owner, Manager, Cashier
CRUD	/addon-groups, /stations, /payment-methods	Add-ons, stations, payment methods	Owner, Manager
POST	/items/import, GET /items/export	CSV import/export	Owner, Manager
CRUD	/table-sections, /tables	Floor and tables	Owner, Manager
GET	/pos/bootstrap?branch_id=	Menu, tables, settings, payment methods in one call (for caching)	Staff

9.3 Orders, KOT and billing

Method	Path	Purpose	Roles
POST	/orders	Create order (client UUID id) with items	Waiter+
GET	/orders, /orders/{id}	List with filters; detail	Staff
POST	/orders/{id}/items	Add items	Waiter+
PATCH/DELETE	/orders/{id}/items/{itemId}	Edit qty, void (rules in 6.3)	Waiter+ / approval
POST	/orders/{id}/send-kot	Create KOT batch from unsent items	Waiter+
POST	/orders/{id}/transfer, /merge, /split	Table transfer, merge, split bill	Cashier+
POST	/orders/{id}/discount	Apply discount with reason	Cashier+ (limit)
POST	/orders/{id}/request-bill	Set bill_requested	Waiter+
GET	/orders/{id}/bill	Calculated bill breakdown	Cashier+
POST	/orders/{id}/pay	Add payment(s); closes order when fully paid	Cashier+
POST	/orders/{id}/cancel	Cancel with reason (approval after KOT)	Manager+
POST	/orders/{id}/refund	Refund with reason	Manager+
GET	/orders/{id}/receipt, /kots/{id}/print	Print data (also reprint, audited)	Staff
GET	/kots?status=&station_id=	KDS list	Kitchen+
PATCH	/kots/items/{id}/status	Update item status	Kitchen+
POST	/approvals, POST /approvals/{id}/approve	Manager approval requests	Staff / Manager
POST	/sync/batch	Replay offline outbox (array of operations with idempotency keys)	Staff

9.4 Shifts, inventory and purchasing

Method	Path	Purpose	Roles
POST	/shifts/open, /shifts/{id}/close	Open/close shift	Cashier+
POST	/shifts/{id}/cash-movements	Cash in/out	Cashier+
GET	/shifts, /shifts/{id}/report	History and shift report	Manager+
CRUD	/inventory-items, /recipes	Ingredients and recipes	Manager+
GET	/stock?branch_id=	Current stock and valuation	Manager+
POST	/stock/adjust, /stock/wastage, /stock/transfer	Manual movements (reason required)	Manager+
POST	/stock-counts, /stock-counts/{id}/finalize	Physical stock count	Manager+
CRUD	/suppliers, /purchase-orders	Purchasing	Manager+
POST	/purchase-orders/{id}/receive	Receive goods (stock in, payable)	Manager+
CRUD	/expenses	Operating expenses	Manager+
CRUD	/customers	Customer directory	Cashier+

9.5 Reports, audit and platform admin

Method	Path	Purpose	Roles
GET	/reports/dashboard	KPI cards and charts	Owner, Manager
GET	/reports/sales, /reports/items, /reports/staff	Sales analytics with group_by and date range	Owner, Manager
GET	/reports/profit-loss	Revenue, COGS, expenses, net profit	Owner, Manager (branch)
GET	/reports/exceptions	Discounts, voids, refunds, reprints	Owner, Manager
GET	/reports/stock, /reports/tax, /reports/cash	Stock, VAT and cash reports	Owner, Manager
GET	/reports/{name}/export?format=csv|pdf	Export	Owner, Manager
GET	/audit-logs	Search audit trail	Owner, Manager
GET	/subscription	Current plan, trial days, usage	Owner
GET/POST/PATCH	/admin/restaurants, /admin/plans, /admin/subscriptions	Super admin console endpoints	Super Admin
POST	/admin/restaurants/{id}/impersonate	Audited support access	Super Admin
GET	/health, /ready	Liveness and readiness	Public

10. Frontend Specification

10.1 Applications inside one React project

Area	Route prefix	Description
POS terminal	/pos	Table map, order screen, bill and payment screen, shift screen, held orders
Waiter mode	/waiter	Compact mobile view: tables, quick order, send to kitchen, ready notifications
Kitchen display	/kds	Full-screen ticket board by station, bump buttons, timers, sound
Back office	/admin	Dashboard, menu, tables, inventory, purchasing, staff, reports, settings, audit
Platform admin	/platform	Super admin console for tenants, plans and subscriptions
Auth	/login, /register	Email login, PIN pad, registration with trial start

10.2 Key screens

- **Table map**: color-coded tables (free, occupied, bill requested, reserved), tap to open order, drag to transfer.
- **Order screen**: left category tabs and search, center item grid with images, right cart with qty steppers, notes, discount, Send to Kitchen and Bill buttons; sticky totals.
- **Payment screen**: numeric keypad, quick cash amounts, split tender, change due, Print and New Order actions.
- **KDS**: columns New, Preparing, Ready; large type; elapsed timer turns amber then red; tap to advance status.
- **Dashboard**: KPI cards (sales, orders, avg bill, net profit), sales trend, top items, payment mix, low stock list.
- **Menu manager**: drag-sort categories, item form with variants, add-ons, recipe tab and availability switch.
- **Inventory**: stock table with filters, movement history, wastage and adjustment dialogs, stock count wizard.
- **Reports**: date presets, group-by selector, table + chart, export buttons.
- **Sync center**: pending outbox, failed operations with retry/resolve.

10.3 Frontend structure

```
frontend/
  src/
    app/            routes, providers, layouts, guards (role-based)
    features/
      auth/ pos/ waiter/ kds/ menu/ tables/ inventory/ purchasing/
      reports/ staff/ settings/ shifts/ subscription/ platform/
    components/     ui primitives (shadcn), keypad, receipt, charts
    lib/            api client (axios/fetch + interceptors), ws client, money utils, i18n
    offline/        dexie db, outbox queue, sync engine, connectivity hook
    locales/        en.json, bn.json
    print/          receipt58.css, receipt80.css, kot templates, print-agent client
  public/           manifest.webmanifest, icons
```

10.4 UX requirements

- Minimum tap target 44px; primary actions reachable by thumb on a tablet.
- Optimistic UI for adding items; never block the cashier waiting for the network when offline mode is enabled.
- Confirmation only for destructive actions; undo toast for item removal before KOT.
- All money displayed through a single formatter using restaurant currency and locale.
- Accessibility: labels, focus states, keyboard navigation on desktop, color is never the only status indicator.

11. Backend Project Structure

```
backend/
  cmd/api/main.go                entrypoint
  cmd/worker/main.go             background jobs
  internal/
    config/                      env loading and validation
    platform/ (db, logger, ws, mailer, sms, storage, clock)
    middleware/                  auth, tenant, rbac, ratelimit, requestid, recover
    auth/ users/ restaurants/ branches/ settings/
    menu/ tables/ customers/
    orders/ kot/ billing/ payments/ shifts/
    inventory/ recipes/ purchasing/ expenses/
    reports/ audit/ subscription/ admin/ sync/
    printing/                    receipt and KOT data builders
  migrations/                    0001_init.up.sql ... (golang-migrate)
  queries/                       sqlc query files
  sqlc.yaml
  Makefile  Dockerfile
print-agent/                     optional Go binary for ESC/POS
deploy/ docker-compose.yml, nginx.conf, .env.example
docs/ decisions.md, api.openapi.yaml, runbook.md
```

12. Offline Sync Design

- **Client IDs**: orders, order_items, payments and KOT batches use client-generated UUIDs so they can be created offline.
- **Outbox**: each mutation is stored in IndexedDB as {id, type, payload, idempotency_key, created_at, status}. The sync engine sends them in creation order to POST /sync/batch when online.
- **Idempotency**: server stores idempotency_key results in outbox_sync_log; replays return the stored response.
- **Number allocation**: offline receipts show a temporary local number (e.g., L-0001); the server assigns the final invoice_no on sync. Receipts printed offline carry the local number and are reconciled later.
- **Conflicts**: server wins for menu and price data; for orders, last-write per field with a conflict record visible in Sync center. Paid orders are never overwritten.
- **Cache refresh**: /pos/bootstrap is fetched on login, on reconnect and every 10 minutes while online; menu.availability events patch the cache live.
- **Scope**: offline mode applies to POS and waiter flows; back-office screens require connectivity.

13. Non-Functional Requirements

Area	Requirement
Performance	p95 API latency under 300 ms for POS endpoints on a 2 vCPU server; dashboard queries under 1.5 s for 1 year of data per branch.
Scalability	Support 1,000 restaurants and 200 concurrent WebSocket clients per instance in the first deployment; stateless API for horizontal scaling.
Availability	Target 99.5%; graceful degradation to offline mode in the client.
Data integrity	ACID transactions for orders, payments, stock; foreign keys and check constraints; no float money.
Backup	Daily PostgreSQL backup with 14-day retention and documented restore procedure; test restore monthly.
Security	OWASP ASVS L2 mindset; dependency scanning in CI; HTTPS only; audit trail; least-privilege DB user.
Observability	Structured JSON logs with request IDs, /health and /ready, Prometheus metrics endpoint, error tracking hook.
Compatibility	Latest Chrome/Edge/Safari, Android tablets and iPad; 58mm and 80mm thermal printers.
Maintainability	Lint (golangci-lint, ESLint), formatting, typed API client generated from OpenAPI, README per module.
Localization	All user-facing strings externalized; Bangla and English complete before release.

14. Phased Implementation Plan

Implement phases strictly in order. Each phase ends with a demo-able build, passing tests, updated docs and a tagged release. Do not skip acceptance checks.

Phase 0 - Foundation (week 1)

- Monorepo setup, Docker compose (Postgres, Redis, API, web), CI pipeline, lint and test jobs.
- Migration framework, base schema (restaurants, branches, users, settings), config loader, logger, error format.
- Auth (register, login, refresh), JWT middleware, RBAC, tenant middleware, audit helper.
- React shell: routing, layouts, API client, i18n, theme, role guards.

> **Acceptance:** registering creates a restaurant, owner and trial; login works; tenant isolation test passes; CI green.

Phase 1 - MVP billing (weeks 2-4)

- Menu: categories, items, variants, add-ons, availability; tables and sections.
- POS: order creation, add/remove items, discounts, bill calculation, split tender payment, receipt print (58/80mm CSS).
- Order numbering and gap-free invoices; take-away token numbers.
- Basic reports: daily sales, payment mix, item sales; staff management.
- Shift open/close with cash reconciliation.

> **Acceptance:** a cashier can create a dine-in order, add items, apply a discount, take cash + bKash payment, print a receipt and see the sale in the daily report; totals match Appendix A test vectors; shift variance computed correctly.

Phase 2 - Kitchen and operations (weeks 5-6)

- KOT creation with batches, KOT print, Kitchen Display (WebSocket), status updates, late-ticket highlight and sound.
- Table status live updates, transfer, merge, split bill; approvals for post-KOT voids and high discounts.
- Waiter mobile view with ready notifications; audit log viewer; exceptions report.

> **Acceptance:** two browsers (POS and KDS) show a new order within 1 second; item status changes propagate to the waiter; voiding after KOT requires manager approval and appears in the exceptions report.

Phase 3 - Inventory and profit (weeks 7-9)

- Inventory items, units, recipes, branch stock, stock movements ledger, automatic deduction on payment.
- Suppliers, purchase orders, receiving with weighted-average cost; wastage, adjustments, stock count.
- Expenses; profit and loss report; item margin report; low-stock job and alerts; owner dashboard KPIs.

> **Acceptance:** selling 10 burgers deducts exact recipe quantities; receiving stock updates average cost; profit report equals Revenue - COGS - Expenses for seeded data; refund with restock restores quantities.

Phase 4 - Platform, multi-branch, offline (weeks 10-13)

- Plans, subscriptions, trial/grace/suspension logic, super admin console, in-app banners.
- Multi-branch features: branch switcher, branch overrides, stock transfer, consolidated reports.
- PWA offline mode: cached bootstrap, outbox, sync batch endpoint, sync center, connectivity indicator.
- Local print agent with ESC/POS, cash drawer kick, multiple printers; Bangla/English receipts.

> **Acceptance:** with network disabled the cashier can create, pay and print orders; on reconnect all orders appear on the server once (idempotent) with correct totals; expired trial moves the tenant to read-only; super admin can extend it.

Phase 5 - Hardening and launch (weeks 14-15)

- Load test (k6) on POS endpoints and WebSocket; fix hotspots and add indexes.
- Security review, dependency audit, backup and restore drill, runbook, monitoring dashboards.
- Seed demo restaurant, onboarding wizard, user documentation (EN/BN), release v1.0.

15. Testing and Quality

Level	Scope	Tools
Unit (Go)	Bill calculation, discounts, rounding, state machines, permission checks, cost averaging	go test, testify
Integration (Go)	Repositories and transactions against real Postgres; tenant isolation; invoice counter concurrency	testcontainers-go
API contract	Every endpoint vs OpenAPI; error formats; RBAC matrix	schemathesis or custom
Frontend unit	Money utils, cart store, offline queue, hooks	Vitest, Testing Library
End to end	Order to payment to report; KOT to KDS; offline to sync; trial expiry	Playwright
Performance	POS order creation and WebSocket fan-out	k6
Security	AuthZ bypass attempts, SQL injection, rate limit, token reuse	Manual checklist + automated scans

Minimum coverage targets: 80% on billing, inventory and payment packages; every bug fix adds a regression test.

16. Deployment and Configuration

16.1 Environment variables

```
APP_ENV=production
HTTP_PORT=8080
DATABASE_URL=postgres://user:pass@db:5432/restoflow?sslmode=disable
REDIS_URL=redis://redis:6379/0
JWT_ACCESS_SECRET=change-me
JWT_REFRESH_SECRET=change-me
ACCESS_TTL=15m
REFRESH_TTL=720h
CORS_ORIGINS=https://app.example.com
FILE_STORAGE=local|s3
S3_BUCKET= S3_REGION= S3_KEY= S3_SECRET=
SMTP_HOST= SMTP_USER= SMTP_PASS=
SMS_PROVIDER= SMS_API_KEY=
TRIAL_DAYS=14
DEFAULT_CURRENCY=BDT
DEFAULT_TIMEZONE=Asia/Dhaka
```

16.2 Delivery steps

- docker-compose up for local development with seeded demo data (make seed).
- Production: Docker images for api, worker, web (Nginx), managed PostgreSQL, TLS via Let's Encrypt.
- Migrations run automatically on deploy with a lock; rollback scripts for each migration.
- CI/CD: lint, test, build images, push, deploy to staging, smoke test, manual promote to production.
- Provide docs/runbook.md: backup, restore, rotate secrets, scale workers, handle a stuck sync queue.

17. Definition of Done and Seed Data

- All Must (M) requirements implemented and covered by automated tests; Should (S) items implemented or documented as deferred.
- No critical or high issues open; tenant isolation and RBAC tests pass; load test targets met.
- README, OpenAPI file, runbook and user guide complete; Bangla and English strings complete.
- Seed script creates: demo restaurant, 2 branches, 6 staff (one per role), 5 categories, 40 items with variants, 20 tables, 30 inventory items with recipes, suppliers, 30 days of sample orders and expenses for dashboards.

Appendix A. Calculation Rules and Test Vectors

A.1 Bill calculation (exclusive tax mode)

```
line_total      = (unit_price + sum(addon_price)) * qty - line_discount
subtotal        = sum(line_total)
order_discount  = percent ? round(subtotal * pct / 100) : fixed   (never above subtotal)
taxable_base    = subtotal - order_discount
service_charge  = round(taxable_base * service_pct / 100)
vat             = round((taxable_base + service_charge) * vat_pct / 100)   [configurable base]
delivery_fee    = as entered (delivery orders)
pre_round_total = taxable_base + service_charge + vat + delivery_fee
total           = round_to_nearest(pre_round_total, rounding_unit)   -> round_off = total - pre_round_total
change_due      = max(0, cash_received - amount_due_in_cash)
```

Inclusive tax mode: prices already contain VAT; vat = round(base * vat_pct / (100 + vat_pct)) and is shown for information. All rounding uses half-up on integer minor units.

A.2 Test vectors (BDT, amounts in taka for readability)

Case	Inputs	Expected
1	2 x 350 + 1 x 120; discount 0; service 5%; VAT 15%	subtotal 820; service 41; VAT 129.15 -> 129; total 990
2	Subtotal 1,000; discount 10%; service 10%; VAT 15%	base 900; service 90; VAT 148.5 -> 149 (half-up); total 1,139
3	Subtotal 500; fixed discount 600	Discount capped to 500; total 0 (requires manager approval)
4	Split tender: total 990 = cash 500 + bKash 490	Order paid; change 0; two payment rows

A.3 Profit and loss

```
Revenue (net)  = sum(order totals) - VAT - refunds               (service charge counted as revenue)
COGS           = sum(order_item qty * cost_minor_snapshot) + wastage value
Gross profit   = Revenue (net) - COGS
Net profit     = Gross profit - operating expenses (rent, salary, utilities, other)
Margin %       = Net profit / Revenue (net) * 100
Cost snapshot  = weighted-average cost of recipe ingredients at time of sale
```

A.4 Weighted average cost

```
new_avg_cost = (old_qty * old_avg_cost + received_qty * unit_cost) / (old_qty + received_qty)
Sales and wastage consume at the current avg_cost; transfers move stock at avg_cost.
```

A.5 Receipt layout (80mm / 58mm)

- Header: logo, restaurant name, branch address, phone, VAT/BIN number.
- Meta: invoice no, date and time, order type, table or token, cashier, waiter.
- Lines: item name (and variant/add-ons), qty, unit price, line total.
- Totals: subtotal, discount, service charge, VAT, delivery fee, round-off, grand total, payments, change.
- Footer: custom message, QR code (optional), software credit line (configurable).